

ECE 473/573
Cloud Computing and Cloud Native Systems
Lecture 11 Cloud Native Patterns

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

September 23, 2026

Outline

Cloud Native Patterns

The Context Package

Decorator and Retry

Concurrency and Fan-In

Reading Assignment

- ▶ This lecture: 4
- ▶ Next two lectures: Introduction to Cloud Security

Outline

Cloud Native Patterns

The Context Package

Decorator and Retry

Concurrency and Fan-In

Fallacies of Distributed Computing

- ▶ The network is reliable
- ▶ Latency is zero
- ▶ Bandwidth is infinite
- ▶ The network is secure
- ▶ Topology doesn't change
- ▶ There is one administrator
- ▶ Transport cost is zero
- ▶ The network is homogeneous
- ▶ Services are reliable

Cloud Native Patterns

- ▶ Patterns are proven development paradigms.
 - ▶ Learn from professionals.
 - ▶ Facilitate communication between professionals.
- ▶ Cloud native patterns help to address the fallacies of distributed computing and mitigate them.
- ▶ Explore unique and novel Go implementations.
 - ▶ Combining classical design patterns with goroutines and channels.

Outline

Cloud Native Patterns

The Context Package

Decorator and Retry

Concurrency and Fan-In

The Context Package

- ▶ Introduced in Go 1.7
- ▶ Provides a unified framework for carrying deadlines, cancellation signals, and request-scoped values between function calls and threads.
- ▶ Via the `context.Context` interface.

The Context Interface

```
type Context interface {  
    Done() <-chan struct{}  
    Err() error  
    Deadline() (deadline time.Time, ok bool)  
    Value(key interface{}) interface{}  
}
```

- ▶ A context could refer to a request from a user.
- ▶ **Done** returns a channel indicating if the context is cancelled.
 - ▶ The channel is closed when the context is cancelled.
 - ▶ The channel is not used to communicate any data (**struct{}**).
 - ▶ **Err** indicates the reason of cancellation.
- ▶ **Deadline** returns the deadline of the context if there is any.
 - ▶ A function could choose to terminate if it decides there is not enough time left.
- ▶ **Value** allows to access data associated with this context organized as key-value pairs.
 - ▶ E.g. the identity of the user

What Context Can Do for You

- ▶ Consider a RESTful request that need to be handled.
 - ▶ Similar to our simple key-value store example, this request will go through multiple functions and threads, or even another service, before a response can be generated.
- ▶ What if the request is cancelled during the process?
 - ▶ Completing the process but not returning the response will work but be a waste.
 - ▶ For efficiency, need a mechanism to communicate to functions and threads that the request is cancelled.
- ▶ Context works as a mechanism to notify so.
 - ▶ Functions and threads can check if `Done` returns a closed channel and terminate without consuming additional resources.
 - ▶ Context has to be thread safe for such purpose – be careful with the `Value` method to not modify what it returns.

Building Context

```
func Background() Context
func TODO() Context

// manually cancel via CancelFunc
func WithCancel(parent Context) (Context, CancelFunc)

// automatically cancel after duration or at deadline,
// or manually cancel via CancelFunc
func WithTimeout(parent Context, time.Duration) (Context, CancelFunc)
func WithDeadline(parent Context, time.Time) (Context, CancelFunc)

// create or update data as key/value pairs
func WithValue(parent Context, key, val interface{}) Context
```

- ▶ Start with the empty context either from [Background](#) or [TODO](#)
- ▶ Add cancellations, deadlines, and data by calling the other functions.
- ▶ This is also known as the Decorator pattern.
 - ▶ Widely used to dynamically change object behaviors by introducing new objects without modifying existing objects.
 - ▶ So that objects can be shared by goroutines (and threads) without using locks.

An Example

```
// ReadObject need to follow ctx correctly,  
// and it may create additional contexts for any function it uses.  
func ReadObject(ctx context.Context, out chan<- Value) error {  
    dctx, cancel := context.WithTimeout(ctx, time.Second * 10)  
    defer cancel() // always cancel at the end  
    res, err := RequestObject(dctx) // set a timeout of 10 seconds  
    if err != nil { // if RequestObject fails or times out  
        return err  
    }  
    for { // it takes time to send response back  
        select {  
        case out <- res: // receive from res; send to out  
        case <-ctx.Done(): // if ctx is cancelled somewhere,  
            return ctx.Err() // ReadObject should terminate immediately  
        }  
    }  
} // adapted from textbook example by giving meaningful names
```

- ▶ **ReadObject** need to send the object to **out**.
 - ▶ It calls **RequestObject** to obtain a channel to receive it.
- ▶ However, **RequestObject** will need to obtain the object from another service, which may be too busy sometimes.
 - ▶ **ReadObject** sets a timeout for **RequestObject**.

Outline

Cloud Native Patterns

The Context Package

Decorator and Retry

Concurrency and Fan-In

Retry

- ▶ Errors and failures are always there.
 - ▶ Some of them are transient due to temporary conditions.
- ▶ Transient faults typically resolve themselves after a bit of time.
- ▶ Retry: retrying a failed operation that is most likely transient.
- ▶ Participants
 - ▶ Effector: the original function interacting with the service.
 - ▶ Retry: a closure with the same function signature as Effector that retries Effector if failed.
 - ▶ Retry works as a decorator of Effector to change the behavior.
- ▶ Use this pattern instead of adding a loop (and logging) to any function you would like to retry.

Retry Example

```
type Effector func(context.Context) (string, error)
func Retry(effector Effector, retries int, delay time.Duration) Effector {
    return func(ctx context.Context) (string, error) {
        for r := 0; ; r++ {
            response, err := effector(ctx)
            if err == nil || r >= retries {
                return response, err
            }
            log.Printf("Attempt %d failed; retrying in %v", r + 1, delay)
            select {
                case <-time.After(delay):
                case <-ctx.Done():
                    return "", ctx.Err()
            }
        }
    }
}
```

- ▶ Decorate `effector` by using an anonymous function.
- ▶ Use `select` to wait for delay and cancellation at the same time – whichever comes first will be handled.

Retry Example (Cont.)

```
func main() {
    // a service that fails frequently
    count := 0
    mockService := func(ctx context.Context) (string, error) {
        count++
        if count%3 != 0 {
            return "", errors.New("temporary failure")
        }
        return "success!", nil
    }

    // up to 5 retries with 1s delay
    retryMockService := Retry(mockService, 5, time.Second)

    result, err := retryMockService(context.Background())
    if err != nil {
        fmt.Println("Final error:", err)
        return
    }
    fmt.Println("Got result:", result)
}
```

Outline

Cloud Native Patterns

The Context Package

Decorator and Retry

Concurrency and Fan-In

Fan-In

- ▶ To utilize multiple CPU cores to process large amount of data, multiple worker goroutines are needed.
 - ▶ They can read the same channel for jobs distributed to them.
- ▶ What if they need to send back results?
 - ▶ Via channels for both data and completion.
 - ▶ `select` allows to wait on a predefined list of channels but not an array of channels.
- ▶ Fan-in: multiplex input channels onto one output channel.
 - ▶ Workers cannot write to the output channel directly as they need their own input channels to signal completion.
- ▶ Participants
 - ▶ Sources: inputs channels of the same type.
 - ▶ Destination: output channel with the same type as Sources.
 - ▶ Funnel: take Sources and return Destination, output any from Sources to Destination.

Funnel Implementation

```
func Funnel(sources ...<-chan int) <-chan int {
    dest := make(chan int) // The shared output channel
    var wg sync.WaitGroup // Used to automatically close dest
                          // when all sources are closed
    wg.Add(len(sources)) // Set size of the WaitGroup
    for _, ch := range sources { // Start a goroutine for each source
        go func(c <-chan int) {
            defer wg.Done() // Notify WaitGroup when c closes
            for n := range c {
                dest <- n
            }
        }(ch)
    }
    go func() { // Start a goroutine to close dest
        wg.Wait() // after all sources close
        close(dest)
    }()
    return dest
}
```

- `sync.WaitGroup` manages a count of source channels that are not closed yet.

Fan-In Example

```
func main() {  
    sources := make([]chan int, 0) // Create an empty channel slice  
    for i := 0; i < 3; i++ {  
        ch := make(chan int)  
        sources = append(sources, ch) // Create a channel; add to sources  
        go func() { // Run a toy goroutine for each worker  
            defer close(ch) // Close ch when the routine ends  
            for i := 1; i <= 5; i++ {  
                ch <- i  
                time.Sleep(time.Second)  
            }  
        }()  
    }  
    dest := Funnel(sources...)  
    for d := range dest {  
        fmt.Println(d)  
    }  
}
```

- No need to use `sync.WaitGroup` in `main`.

Summary

- ▶ Distributed computing is very different than simply connecting services via networks.
- ▶ Learn proven paradigms for distributed computing via cloud native patterns.